

Técnicas Computacionais Avançadas

Lista 01 — Algoritmos de Ordenação

Do algoritmo ao experimento

Implementei e comparei seis estratégias de ordenação em **Python** e **C++**.

- Dados reproduzíveis, sete tamanhos e sete famílias de entrada;
- Medições separadas de tempo, memória e operações elementares;
- Visualização passo a passo para tornar as estratégias observáveis.

Extensão da atividade

O projeto estende propositalmente o pedido da lista para investigar:

- memória, além do tempo e das operações;
- diferenças entre C++ e Python;
- uma visualização HTML do passo a passo dos algoritmos.

A intenção é manter essa análise comparativa nas tarefas posteriores da disciplina.

Algoritmos implementados

Método	Resumo
Selection	Seleciona o menor elemento restante; $O(n^2)$.
Insertion	Insere a chave; próximo de $O(n)$ em dados ordenados.
Merge Smarter	Intercala com buffer reutilizado; $O(n \log n)$.
Quick Classic	Primeiro pivô; pode degradar em entradas ordenadas.
Quick Smarter	Pivô pseudoaleatório e recursão limitada.
Radix Decimal	LSD sem comparações entre elementos.

Visualização das estratégias

[Abrir a visualização comparativa interativa](#)

Os seis métodos ordenam a mesma entrada de 16 elementos, com valores repetidos. A animação é didática e não integra o benchmark.

Famílias de entrada

Família	Construção	O que ela evidencia
<code>uniform_random</code>	Valores quantizados.	Caso geral.
<code>sorted</code> , <code>reverse_sorted</code> , <code>nearly_sorted</code> , <code>nearly_reverse_sorted</code>	Ordem crescente, decrescente ou com pequenas trocas.	Adaptação à ordem inicial e escolha do pivô.
<code>many_repeated</code> , <code>all_equal</code>	Muitas chaves repetidas ou uma única chave.	Tratamento de repetições.

Campanha de análise

7 tamanhos × 7 famílias × 5 repetições = 245 casos

Tamanhos: 1, 10, 10^2 , 10^3 , 10^4 , 10^5 e 10^6 .

245 casos × 6 métodos × 2 backends × 3 medições = 8.820 execuções

Os resultados são persistidos incrementalmente, permitindo retomar uma campanha interrompida.

Métricas de análise

- **Tempo:** média de `elapsed_seconds` em cinco repetições.
- **Memória:** média do pico adicional de memória.
- **Telemetria:** `comparisons`, `swaps` e `writes`.

As medições são independentes para não instrumentar o tempo normal. Linhas contínuas representam Python; tracejadas, C++.

Análise dos resultados — efeito da entrada

- **Selection** mantém aproximadamente $n(n - 1)/2$ comparações: é o mais lento de forma sistemática e quase não reage à organização inicial.
- **Insertion** é o mais rápido em `sorted`, `nearly_sorted` e `all_equal`; em entradas aleatórias ou inversas, os deslocamentos o levam de volta ao comportamento $O(n^2)$.
- **Quick Classic** é rápido em `uniform_random`, mas a escolha fixa do primeiro pivô gera partições degeneradas em entradas ordenadas, inversas e próximas desses casos.
- **Quick Smarter** e **Merge Smarter** são os métodos mais regulares entre as famílias; a estratégia de pivô e recursão torna o Quick Smarter robusto onde o Classic falha.

Análise dos resultados — recursos e implementação

- **Memória:** Radix apresentou o maior pico adicional em todas as famílias, seguido pelo Merge; ambos usam estruturas auxiliares proporcionais a n .
- **Python × C++:** o C++ foi mais rápido em todas as comparações equivalentes concluídas. Para $n = 100.000$, o ganho mediano variou de aproximadamente **66×** (Selection) a próximo de **1.000×** (Insertion em casos custosos).
- **Falhas são resultado:** no Quick Classic, a recursão degenerada causou `RecursionError` em Python e *stack overflow* em C++; não são pontos omitidos dos gráficos.

Não existe algoritmo universalmente superior: a forma da entrada, a memória disponível e o custo por operação da implementação mudam a escolha.

Selection Sort

C++

```
template <typename ProbeType>
void selection_sort_impl(std::span<double> values, ProbeType& probe) {
    for (std::size_t index_i = 0; index_i < values.size(); ++index_i) { // i=(0)..(n-1)
        std::size_t marker = index_i; // m=i

        for (std::size_t index_j = index_i + 1; index_j < values.size();
              ++index_j) { // j=(i+1)..(n)
            if (probe.lt(values[index_j], values[marker])) { // se x_j < x_m
                marker = index_j; // m=j
            }
        }

        probe.swap(values, index_i, marker); // swap(x_i, x_m)
    }
}
```

Python

```
def selection_sort(
    values,
    metrics: Metrics | None = None,
    trace: Trace | None = None,
) -> None:
    probe = make_probe(metrics, trace)

    for index_i in range(len(values) - 1): # i=(1)..(n-1)
        marker = index_i # m=i

        probe.event(
            "select_minimum",
            indices=(marker,),
            values=(values[marker],),
        ) # mínimo atual m=i

        for index_j in range(index_i + 1, len(values)): # j=(i+1)..(n)
            if probe.lt(
                values[index_j],
                values[marker],
                indices=(index_j, marker),
                roles=("current", "marker"),
            ): # se x_j < x_m
                marker = index_j # m=j

            probe.event(
                "select_minimum",
                indices=(marker,),
                values=(values[marker],),
            ) # novo mínimo m=j
```

Insertion Sort

C++

```
template <typename ProbeType>
void insertion_sort_impl(std::span<double> values, ProbeType& probe) {
    for (std::size_t index_i = 1; index_i < values.size(); ++index_i) { // i=(1)..(n-1)
        const double value_marker = values[index_i]; // v = x_i
        std::size_t index_j = index_i; // j = i

        while (index_j > 0 &&
            probe.lt(value_marker, values[index_j - 1]) // enquanto x_{j-1} > v
        ) {
            probe.write(values, index_j, values[index_j - 1]); // x_j = x_{j-1}
            --index_j; // j = j - 1
        }

        probe.write(values, index_j, value_marker); // x_j = v
    }
}
```

Python

```
def insertion_sort(
    values,
    metrics: Metrics | None = None,
    trace: Trace | None = None,
) -> None:
    probe = make_probe(metrics, trace)

    for index_i in range(1, len(values)): # i=(1)..(n-1)
        value_marker = values[index_i] # v = x_i
        index_j = index_i # j = i

        probe.event(
            "select_key",
            indices=(index_i,),
            values=(value_marker,),
        ) # seleciona v = x_i

        while index_j > 0 and probe.lt(
            value_marker,
            values[index_j - 1],
            indices=(index_i, index_j - 1),
            roles=("key", "current"),
        ): # enquanto x_{j-1} > v
            probe.write(values, index_j, values[index_j - 1]) # x_j = x_{j-1}
            index_j -= 1 # j = j - 1

        probe.write(values, index_j, value_marker) # x_j = v
```

Merge Sort — Separação

C++

```
template <typename ProbeType>
void merge_sort_recursive(std::span<double> values, std::span<double> buffer,
    std::size_t start, std::size_t end, ProbeType& probe,
    const tca::algorithms::MergeSortOptions& options) {

    if (end - start < 2) { // se n < 2 então retorne {caso básico da recursão}
        return;
    }

    const std::size_t middle = (start + end) / 2; // m = n/2

    merge_sort_recursive(values, buffer, start, middle, probe,
        options); // mergesort(l, m) {recursão esquerda}

    merge_sort_recursive(values, buffer, middle, end, probe,
        options); // mergesort(r, n-m) {recursão direita}

    if (options.buffer == tca::algorithms::MergeBuffer::Local) {
        std::vector<double> local_buffer(end - start);

        merge(values, std::span<double>{local_buffer}, // aloca buffer local
            start, start, middle, end,
            probe); // {combinação classic}
    } else {
        merge(values,
            buffer, // usa buffer único reutilizado
            0, start, middle, end,
            probe); // {combinação smarter}
    }
}
```

Python

```
def _merge_sort(
    values,
    buffer,
    start: int,
    end: int,
    probe,
    buffer_strategy: str,
) -> None:
    if end - start < 2: # se n < 2 então retorne {caso básico da recursão}
        return

    middle = (start + end) // 2 # m = n/2

    _merge_sort(
        values,
        buffer,
        start,
        middle,
        probe,
        buffer_strategy,
    ) # mergesort(l, m) {recursão esquerda}

    _merge_sort(
        values,
        buffer,
        middle,
        end,
        probe,
        buffer_strategy,
    ) # mergesort(r, n-m) {recursão direita}
```

Merge Sort — Combinação

C++

```
template <typename ProbeType>
void merge(std::span<double> values, std::span<double> buffer,
          std::size_t buffer_offset, std::size_t start, std::size_t middle,
          std::size_t end, ProbeType& probe) {
    for (std::size_t index_k = start; index_k < end; ++index_k) {
        const std::size_t index_buffer = index_k - buffer_offset;

        probe.write(buffer, index_buffer,
                    values[index_k]); // l = x[start..m], r = x[m..end] {separação}
    }

    std::size_t index_l = start; // i = 1
    std::size_t index_r = middle; // j = 1

    for (std::size_t index_k = start; index_k < end; ++index_k) {
        // para k = 1..n {combinação}

        const std::size_t index_buffer_l = index_l - buffer_offset;
        const std::size_t index_buffer_r = index_r - buffer_offset;

        if (index_l >= middle) {
            probe.write(values, index_k,
                        buffer[index_buffer_r]); // x_k = r_j
            ++index_r;                          // j = j+1
        } else if (index_r >= end) {
            probe.write(values, index_k,
                        buffer[index_buffer_l]); // x_k = l_i
            ++index_l;                          // i = i+1
        } else if (probe.lt(buffer[index_buffer_r],
```

Python

```
def _merge(
    values,
    buffer,
    start: int,
    middle: int,
    end: int,
    probe,
    buffer_strategy: str,
) -> None:
    if buffer_strategy == "local":
        working_buffer = [None] * (end - start) # buffer local {classic}
        buffer_offset = start

    else:
        working_buffer = buffer # buffer único reutilizado {smarter}
        buffer_offset = 0

    for index_k in range(start, end):
        index_buffer = index_k - buffer_offset

        probe.write(
            working_buffer,
            index_buffer,
            values[index_k],
            target="buffer",
        ) # l = x[start..m], r = x[m..end] {separação}

    index_l = start # i = 1
    index_r = middle # j = 1

    for index_k in range(start, end): # para k = 1..n {combinação}
```

Quick Sort — Recursão

C++

```
template <typename ProbeType>
void quick_sort_recursive(std::span<double> values, std::size_t index_r,
                          std::size_t index_s, ProbeType& probe,
                          const tca::algorithms::QuickSortOptions& options,
                          tca::PRNG& prng) {
    while (index_r < index_s) { // enquanto s > r
        const std::size_t index_j =
            partition(values, index_r, index_s, probe, options.pivot, prng);

        if (options.recursion == tca::algorithms::QuickRecursion::Classic) {
            if (index_j > index_r) {
                quick_sort_recursive(values, index_r, index_j - 1, probe, options,
                                     prng);
            }

            if (index_j < index_s) {
                quick_sort_recursive(values, index_j + 1, index_s, probe, options,
                                     prng);
            }

            return;
        }

        const std::size_t left_size = index_j - index_r;
        const std::size_t right_size = index_s - index_j;

        if (left_size < right_size) {
            if (index_j > index_r) {
                quick_sort_recursive(values, index_r, index_j - 1, probe, options,
                                     prng);
            }
        }
    }
}
```

Python

```
def _quick_sort(
    values,
    index_r: int,
    index_s: int,
    probe,
    pivot: str,
    recursion: str,
    prng: PRNG,
) -> None:
    while index_r < index_s: # enquanto s > r
        index_j = _partition(values, index_r, index_s, probe, pivot, prng)

        if recursion == "classic":
            _quick_sort(values, index_r, index_j - 1, probe, pivot, recursion, prng)
            _quick_sort(values, index_j + 1, index_s, probe, pivot, recursion, prng)
            return

        left_size = index_j - index_r
        right_size = index_s - index_j

        if left_size < right_size:
            _quick_sort(values, index_r, index_j - 1, probe, pivot, recursion, prng)
            index_r = index_j + 1 # continua iterativamente pela direita

        else:
            _quick_sort(values, index_j + 1, index_s, probe, pivot, recursion, prng)
            index_s = index_j - 1 # continua iterativamente pela esquerda
```

Quick Sort — Particionamento

C++

```
template <typename ProbeType>
std::size_t partition(std::span<double> values, std::size_t index_r,
                    std::size_t index_s, ProbeType& probe,
                    tca::algorithms::QuickPivot pivot, tca::PRNG& prng) {
    const std::size_t index_pivot = choose_pivot(index_r, index_s, pivot, prng);
    probe.swap(values, index_r, index_pivot); // move o pivô para x_r

    const double value_pivot = values[index_r]; // v = x_r
    std::size_t index_i = index_r;             // i = r
    std::size_t index_j = index_s + 1;         // j = s+1

    while (true) { // repita {separação}
        ++index_i; // i = i+1

        while (index_i <= index_s &&
               probe.lt(values[index_i], value_pivot)) { // até x_i >= v
            ++index_i; // i = i+1
        }

        --index_j; // j = j-1

        while (probe.lt(value_pivot, values[index_j])) { // até x_j <= v
            --index_j; // j = j-1
        }

        if (index_j <= index_i) { // até j <= i
            break;
        }

        probe.swap(values, index_i, index_j); // troque x_i com x_j
    }
}
```

Python

```
def _partition(
    values, index_r: int, index_s: int, probe, pivot: str, prng: PRNG
) -> int:
    index_pivot = _choose_pivot(index_r, index_s, pivot, prng)

    probe.event(
        "choose_pivot",
        indices=(index_pivot,),
        values=(values[index_pivot],),
        start=index_r,
        end=index_s,
        strategy=pivot,
    )

    probe.swap(values, index_r, index_pivot) # move o pivô para x_r

    value_pivot = values[index_r] # v = x_r
    index_i = index_r # i = r
    index_j = index_s + 1 # j = s+1

    while True: # repita {separação}
        index_i += 1 # i = i+1

        # até x_i >= v
        while index_i <= index_s and probe.lt(
            values[index_i],
            value_pivot,
            indices=(index_i, index_r),
            roles=("current", "pivot"),
        ):
            index_i += 1 # i = i+1
```


Radix Sort

C++

```
template <typename ProbeType>
void radix_sort_impl(std::span<double> values, ProbeType& probe, int digits) {
    if (digits < 0 || digits > tca::MAX_DECIMAL_DIGITS) {
        throw std::invalid_argument("digits must be between 0 and 15");
    }

    if (values.size() < 2) { // se n < 2 então retorne {caso básico}
        return;
    }

    std::vector<std::int64_t> raw_keys(values.size());

    for (std::size_t index_i = 0; index_i < values.size(); ++index_i) {
        raw_keys[index_i] = tca::decimal_key(values[index_i],
                                             digits); // k_i = trunc(x_i * 10^digits)
    }

    const auto keys =
        normalize_keys(raw_keys, digits); // remove zeros comuns da quantizacao

    const auto minimum_key =
        *std::min_element(keys.begin(), keys.end()); // k_min = min(k)

    std::vector<std::uint64_t> shifted_keys(values.size());

    for (std::size_t index_i = 0; index_i < keys.size(); ++index_i) {
        shifted_keys[index_i] =
            // keys[index_i] - minimum_key seria um problema pois
            // se keys[index_i] for INT64_MAX e minimum_key for INT64_MIN,
            // a diferença não cabe em int64_t
            // Logo, a aritmética é feita em uint64_t.
        shifted_keys[index_i] = static_cast<uint64_t>(keys[index_i] - minimum_key);
    }
}
```

Python

```
def radix_sort(
    values,
    metrics: Metrics | None = None,
    trace: Trace | None = None,
    *,
    digits: int = 3,
) -> None:
    if not isinstance(digits, int):
        raise TypeError("digits must be an integer")

    if not 0 <= digits <= MAX_DECIMAL_DIGITS:
        raise ValueError(f"digits must be between 0 and {MAX_DECIMAL_DIGITS}")

    probe = make_probe(metrics, trace)

    if len(values) < 2: # se n < 2 então retorne {caso básico}
        return

    raw_keys = [
        decimal_key(value, digits) for value in values
    ] # k_i = trunc(x_i * 10^digits)

    keys = _normalize_keys(
        raw_keys,
        digits,
    ) # remove zeros comuns introduzidos pela quantização

    minimum_key = min(keys) # k_min = min(k)
    shifted_keys = [key - minimum_key for key in keys] # y_i = k_i - k_min
    indices = list(range(len(values))) # idx = [0, 1, ..., n-1]
```